

Reviewer Pack — Minimal Local Index of K-theory over Function Fields vs Reso...

Entry d6670829838b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 18:50:16 UTC

Minimal Local Index of K-theory over Function Fields vs Resolution Proof Width for Tseitin Formulas

Entry ID: d6670829838b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 18:50:16 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-theory (over function fields) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity (for Tseitin Formulas)

Statement:

[‘For every Tseitin formula G with n variables, the local index of its associated ring $R(G)$ is bounded by $O(2^{(n/2)})$ such that the resolution proof width of G is at least $2^{O(n - \log n)}$.’, ‘More formally, for all Tseitin formulas G with n variables, there exist constants $c_1, c_2 > 0$ such that’, ‘ $\text{LocalIndex}(R(G)) \leq c_1 * 2^{(n/2)}$ and $\text{ResolutionProofWidth}(G) \geq 2^{O(n - \log n)}$.’]

Rationale (proposer’s reasoning):

[‘Algebraic K-theory provides a rich framework for studying the arithmetic properties of rings, including those defined over function fields. This conjecture suggests that the complexity of resolving Tseitin formulas is deeply connected to the algebraic structure of their defining rings.’, ‘The local index of a ring measures its complexity in terms of its k-theoretical properties. If this conjecture holds, it would imply a strong connection between algebraic K-theory and computational complexity, potentially revealing new insights into the nature of resolution proofs.’, ‘This connection could also shed light on the inherent difficulty of Tseitin formulas, which are known to require exponential size resolution proofs for certain types of expander graphs.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b65f77c1e23d5fe6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For Tseitin formulas G with n variables, if $\text{LocalIndex}(R(G)) \leq c_1 * 2^{(n/2)}$ AND $\text{ResolutionProofWidth}(G) \geq 2^{O(n - \log n)}$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic K-theory function fields AND resolution proof complexity Tseitin formulas - local index ring $R(G)$ $O(2^{(n/2)})$ AND resolution proof width Tseitin formula $2^{O(n - \log n)}$ - K-theory over function fields AND Tseitin formula resolution proof width bound

Top relevant hits considered: - [http://arxiv.org/abs/1311.1421v3] Multiplicative differential algebraic K-theory and applications - [http://arxiv.org/abs/2104.04021v2] A universal characterization of noncommutative motives and secondary algebraic K-theory - [http://arxiv.org/abs/1811.09564v3] Dévissage for Waldhausen K-theory - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/2105.11641v2] Searches for continuous gravitational waves from young supernova remnants in the early third observing run of Advanced L - [http://arxiv.org/abs/2603.19020v1] GWTC-4.0: Tests of General Relativity. II. Parameterized Tests - [http://arxiv.org/abs/1502.02240v2] On the K-theory of linear groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(n)]
        clauses = []
        for i in range(n):
            clauses.append(f'({variables[i]} v ~{variables[i]})')
        for i in range(1, n):
            clauses.append(f'{variables[0]} v {variables[i]}')
        return ' & '.join(clauses)

    def local_index(ring):
        # Placeholder for actual computation
        return 2 ** (len(ring) / 2)
```

```

def resolution_proof_width(formula):
    # Placeholder for actual computation
    return 2 ** len(formula.split(' v '))

n = random.randint(5, 40)
formula = generate_tseitin_formula(n)
ring = formula.replace(' ', '').replace('v', '1').replace('~', '0')
local_idx = local_index(ring)
resolution_width = resolution_proof_width(formula)

return {
    "metric_name": "LocalIndex vs ResolutionWidth",
    "metric_value": local_idx,
    "instances_tested": 1,
    "conjecture_holds": local_idx <= 2 ** (n / 2) and resolution_width >= 2 ** (n - math.log(n)),
    "counterexample": f"n={n}, LocalIndex={local_idx}, ResolutionWidth={resolution_width}" if not local_idx <= 2 ** (n / 2) and resolution_width >= 2 ** (n - math.log(n))
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(100, 999) for _ in range(30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a4f47ca0.py", line 57, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a4f47ca0.py", line 49, in run_trial

```

    "counterexample": f"n={n}, LocalIndex={local_idx}, ResolutionWidth={resolution_width}" if not conjecture_holds
    ~~~~~

```

NameError: name 'conjecture_holds' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify if the conjecture's conditions are met or not. | next: Re-run the test to ensure it completes successfully and provides results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13143
2	propose	ollama_remote	glm4:latest	0	0	11712
3	preregistration	ollama_remote	glm4:latest	0	0	5959
4	novelty	ollama_remote	glm4:latest	0	0	5140
5	novelty	ollama_remote	glm4:latest	0	0	6293
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15195
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8445
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9856
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7674
10	judge	ollama_remote	glm4:latest	0	0	7875

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 91292 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/d6670829838b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d6670829838b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d6670829838b.tar.gz (if generated)